

COPILOTTRAINING
TECHNICAL BRIEFING

Harness Fundamentals

Getting Inside the Agent Loop

See the five jobs every agent harness does, then take them over in GitHub's harness from your own code.

Extend and Embed · CopilotTraining Tech Talk · 60 min · @github/copilot-sdk 1.0.26

🤔 THE CORE QUESTION

"How do I get inside the agent loop and change how it behaves?"

A harness is just a loop: ask the model, run what it asks for, feed back the result, repeat.

Take over the right job, and the agent sees, uses, and does what your workflow needs.

Developers

Already use Copilot in the editor or CLI and want the agent inside their own tools

Platform engineers

Decide which rules run in every harness and which belong in one app

Architects

Choose when a workflow earns its own agent app and what it may see

~170 lines

pi's entire agent loop, one function you can read on one screen

5 places to step in

Loop, context, tools, control, evidence: every hook or callback fills one

Same loop in Copilot

The SDK starts the Copilot CLI runtime; your code steps in at the same five places

📖 TABLE OF CONTENTS

PART 1

What a Harness Is

Five jobs, read in pi's source

Loop, context, tools, control, and evidence, each on one slide of real code

PART 2

Drive GitHub's Harness

The same five jobs, one quickstart file each

Take over four jobs from your own code and build a docs-only onboarding check

PART 3

Decide What It Sees

One policy, a blind tester, and its evidence

Hide the implementation and let a referee name what the tests found

PART 4

Own It

Surrounding controls, pre-flight, named owner

Share a harness others can run, and know who reads each report

PART 1

What a Harness Is

Read pi as source, one slide per job: loop, context, tools, control, and evidence.

The loop

One function of about 170 lines runs every turn

Tools + control

find hands off to fd; beforeToolCall says no

Evidence

Every step is an event any subscriber can read

```
Code excerpts: pi · earendil-works/pi@f9daca3 · MIT License · trimmed, read as source
↳ 5 jobs: loop · context · tools · control · evidence
```

A harness is a loop with five places to step in

Each job is a point in the loop: the question it answers, and where pi does it

Loop	When does the agent take another turn, and when does it stop?	`runLoop`
Context	What does the model see on this turn?	`transformContext`
Tools	What can the agent do, and how does each action run?	`write · find`
Control	Which calls actually run?	`beforeToolCall`
Evidence	What happened, in a form someone can inspect?	`emit · afterToolCall`

pi is one small open-source harness among many; the five jobs are the part that transfers

That's the whole harness: pi's loop is one function of ~170 lines

pi · packages/agent/src/agent-loop.ts#L188-L337 (trimmed)

```
while (true) {                                                    // LOOP: outer, for follow-ups
  while (hasMoreToolCalls || pendingMessages.length > 0) {        // LOOP: one turn per pass
    // … queued steering messages join currentContext.messages    // CONTEXT
    const message = await streamAssistantResponse(currentContext, config, signal, emit, streamFunction);
    const toolCalls = message.content.filter((c) => c.type === "toolCall");
    if (toolCalls.length > 0) {                                   // TOOLS + CONTROL
      const executedToolBatch =
        message.stopReason === "length"
          ? await failToolCallsFromTruncatedMessage(toolCalls, emit)
          : await executeToolCalls(currentContext, message, config, signal, emit);
    }
    await emit({ type: "turn_end", message, toolResults });       // EVIDENCE
    pendingMessages = (await config.getSteeringMessages?.()) || [];
  }   // … then follow-up messages restart the outer loop, or it breaks
}
```

🔧 Tool calls keep it running

No tool calls ends the turn loop; results go back into context.

🧭 People steer mid-run

Steering joins before the next turn; follow-ups restart it.

📡 Every turn emits an event

Where a UI, log, or budget attaches.

pi rebuilds what the model sees before every model call

pi · packages/agent/src/agent-loop.ts#L398-L404

```
let messages = context.messages;
if (config.transformContext) {
  messages = await config.transformContext(messages, signal);
}
const llmMessages = await config.convertToLlm(messages);
```

✂️ transformContext

Prunes, summarizes, or injects messages before each model call.

🚀 prepareRequest

Runs just before the call and can swap the model or thinking level for the next turn.

📣 declareToolChanges

Tells the model when its tool list changed.

💡 The harness decides the model's view on every turn, so code that controls context shapes the agent's whole view.

A harness can bring its own tools: pi's 'write'

pi · packages/coding-agent/src/core/tools/write.ts (trimmed)

```
const writeSchema = Type.Object({
  path: Type.String({ description: "Path to the file to write (relative or absolute)" }),
  content: Type.String({ description: "Content to write to the file" }),
});
export const writeToolSystemPromptContribution = {
  snippet: "Create or overwrite files",
  guidelines: ["Use write only for new files or complete rewrites."],
} as const;
return withFileMutationQueue(absolutePath, async () => {   // … inside execute(), later in the file
  throwIfAborted();
  await ops.writeFile(absolutePath, content);              // … after ops.mkdir; then returns success
```

🧩 A tool is schema + code

The model fills the schema; the harness runs the code.

🏗️ Built in, no shell

write calls no OS command; pi wrote every line.

🔀 Your rules inside

Queue, abort, and ops: disk, SSH, or anywhere.

🚪 Build it or borrow it: pi's 'find' borrows the fd binary by default; the harness still sees call and result.

Before any tool runs, the loop asks your code: run it or block it?

pi · packages/agent/src/agent-loop.ts#L738-L765 (trimmed)

```
if (config.beforeToolCall) {
  const beforeResult = await config.beforeToolCall(
    { assistantMessage, toolCall, args: validatedArgs, context: currentContext }, signal);
  if (beforeResult?.block) {
    const result = createErrorToolResult(beforeResult.reason || "Tool execution was blocked");
    return { kind: "immediate", result, isError: true };
  }
}
// pi · coding-agent/src/core/agent-session.ts#L652-L682: the product installs its policy
this.agent.beforeToolCall = (context) => this._beforeToolCall(context);
this.agent.afterToolCall = (context) => this._afterToolCall(context);
```

↩️ Blocked? The agent hears why

Your reason goes back to the model; it tries another way.

🔭 Your code sees everything

The tool, its arguments, and the conversation so far.

🎀 Half-written calls never run

If a reply hits the length limit, its tool calls fail.

🧩 pi's own coding agent plugs its add-ons in right here; the loop itself never changes.

pi reports every step as an event any subscriber can read

pi · packages/agent/src/agent-loop.ts#L880-L937 (trimmed)

```
if (config.afterToolCall) {
  const afterResult = await config.afterToolCall(
    { toolCall: prepared.toolCall, args: prepared.args, result, isError, /* … */ }, signal);
  if (afterResult) { result = { ...result, content: afterResult.content ?? result.content /* … */ }; }
}
await emit({ type: "tool_execution_end", toolCallId: finalized.toolCall.id,
  toolName: finalized.toolCall.name, result: finalized.result, isError: finalized.isError /* … */ });
// agent.ts#L275-L278
subscribe(listener: (event: AgentEvent, signal: AbortSignal) => Promise<void> | void): () => void {
  this.listeners.add(listener);
  return () => this.listeners.delete(listener);
}
```

afterToolCall can rewrite
Redact or trim before the model reads it.

Audit-ready events
Tool, call, result, error, and duration.

Anything can listen
subscribe feeds a UI, a log, or a budget.

agent_start · turn_start · message_start/update/end · tool_execution_start/update/end · turn_end · agent_end

PART 2

Drive GitHub's Harness

Same loop; GitHub runs it, and your code steps in at the same five places.

🔁

Same loop, your host

Host = the app you write with the SDK; it steps in at five places

📁

One file per job

01-loop.ts through 05-evidence.ts, one slide each

👀

Fresh-eyes check

A docs-only agent files a friction report

```
@github/copilot-sdk 1.0.26 · model auto · FanHub 0ea9e78 · observed 2026-10-26
↳ 24 tool calls · 0 source reads · 3 friction findings
```

Same five jobs, different code: the SDK has its own names for each

Copilot SDK names in the middle; the pi code you just saw, for comparison, on the right

Loop	The runtime; your app drives createSession, sendAndWait, abort	`pi: runLoop`
Context	systemMessage (append, customize, replace), the prompt, model	`pi: transformContext`
Tools	Built-in tools, availableTools, excludedTools, defineTool	`pi: write · find`
Control	onPermissionRequest, hooks.onPreToolUse, repository hooks	`pi: beforeToolCall`
Evidence	streaming, session.on(...), tool handlers, hook logs	`pi: emit · afterToolCall`

Documented: the SDK talks to the Copilot runtime over JSON-RPC and starts and stops it for you

Your code starts GitHub's loop and decides when it stops

examples/sdk-quickstart/01-loop.ts

```ts
import { CopilotClient, approveAll } from "@github/copilot-sdk";

const client = new CopilotClient();
const session = await client.createSession({ model: "auto", onPermissionRequest: approveAll });

const reply = await session.sendAndWait({ prompt: "How do I run this project? Two sentences." });
console.log(reply?.data.content);

await session.disconnect();
await client.stop();
```

🖥️ new CopilotClient()

Starts the Copilot CLI runtime: GitHub's loop.

⏳ sendAndWait

Runs the loop until the agent is done; returns the reply.

🔴 disconnect + stop

Ends the session and shuts the runtime down.

🔁 Same loop as pi. The SDK hands you the five places to step in.

Your code rewrites one section of what the model is told; the rest stays

examples/sdk-quickstart/02-context.ts (trimmed)

```
const session = await client.createSession({
  model: "auto",
  onPermissionRequest: approveAll,
  systemMessage: {
    mode: "customize",
    sections: { custom_instructions: { action: "append", content:
      "You are a first-day engineer who trusts only written documentation. " +
      "Cite the file behind every step. " +
      "Label any step that comes from code or config instead of docs as (undocumented)." } },
  },
```

systemMessage

GitHub's built-in instructions to the model, in named sections.

mode: customize

Edit one section; every other section stays as GitHub wrote it.

custom_instructions

The section for your rules; here, three sentences appended.

💡 Instructions steer the model; the next two slides enforce limits in code.

Your host adds one tool and allows four; the rest disappear

examples/sdk-quickstart/03-tools.ts (trimmed)

```
const OWNERS: Record<string, string> = { "node/backend/src/routes/": "api-team",
  "node/backend/src/database/": "data-team", "node/frontend/": "web-team" };
const lookupOwner = defineTool("lookup_owner", {
  description: "Return the team that owns a repository path.",
  parameters: { type: "object", properties: { path: { type: "string" } }, required: ["path"] },
  handler: async ({ path }: { path: string }) => { /* … owner of the matching OWNERS prefix */ },
});
const session = await client.createSession({ model: "auto", onPermissionRequest: approveAll,
  availableTools: ["view", "glob", "grep", "lookup_owner"], tools: [lookupOwner] });
```

➕ What: tools

Adds a tool: name, schema, handler in your app.

✅ What: availableTools

An allowlist of names; the rest are hidden.

🎯 When to add one

The answer lives in your systems, outside the repo.

🧭 Narrowing the tool set changes the plan the agent makes, before any permission check runs.

Before the agent reads, writes, or runs a command, your code says yes or no

examples/sdk-quickstart/04-control.ts (trimmed)

```
const onPermissionRequest: PermissionHandler = (request) => {
  // … subject = fullCommandText (shell), path (read), or fileName (write)
  if (request.kind === "read") {
    console.log(`[allow] read ${subject}`);
    return { kind: "approve-once" };
  }
  console.log(`[reject] ${request.kind} ${subject}`);
  return { kind: "reject", feedback: "This session is read-only. Use the view tool to inspect files." };
```

onPermissionRequest

Called before every file read, file write, and shell command.

approve-once or reject

Here: reads are allowed; writes and commands are refused.

feedback

The reason goes to the agent, so it finds an allowed way.

Repo hooks still run

Rules checked into the repo apply too, and a repo no wins.

Your handler holds this app's rules; repo hooks hold the rules every agent must follow (enableFileHooks: true).

Every event in GitHub's loop reaches your code

examples/sdk-quickstart/05-evidence.ts (trimmed)

```
const session = await client.createSession({
  model: "auto", streaming: true, onPermissionRequest: approveAll });

session.on("assistant.message_delta", (e) => process.stdout.write(e.data.deltaContent));
session.on("tool.execution_start", (e) => console.log(`\n[tool] ${e.data.toolName}`));
session.on("tool.execution_complete", (e) => console.log(`[tool done] success=${e.data.success}`));
session.on("session.idle", () => console.log("\n[idle]"));

await session.sendAndWait({
  prompt: "List the files in this folder, then tell me which one is the entry point." });
```

What: session.on

Messages, tool calls, and idle, as they happen.

Why: render, count, store

Feed a UI, a budget, or an audit log.

Like pi's subscribe

session.on is the host end of the stream.

Repository hooks add a second record from outside the app; section 3 joins the two by sessionId.

A docs-only agent found FanHub's Node guides disagree

Host decisions

Clean clone at a pinned commit
A fresh temp folder for every run

Sees docs only
Docs, manifests, Dockerfiles, scripts; src/ hidden

report_friction tool
step, docSays, reality, doc, severity: structured findings

Bounded
80 tool calls, then session.abort(); 20-minute timeout

Friction report → docs owner

confusing: port 3001 vs 5265, Docker or none
README needs Docker on 3001; SETUP runs on 5265

blocker: Node 24 and better-sqlite3
No prebuilt binary; docs name no tested Node version

confusing: no fallback for native installs
The agent recovered with npm exec --package=node@20

✅ The agent reports friction; the docs owner decides the fixes.

PART 3

Decide What It Sees

Same host, different view: one visibility policy produces a blind test suite and a referee's verdict.



Two checkpoints

onPreToolUse and onPermissionRequest share a policy



Blind test writer

Contract only, implementation hidden, host reruns



Evidence

JSONL from three layers, joined by sessionId

```
host rerun · TAP output · read-only referee session with BUGS.md
↳ 10 passing · 2 failing · 1 undocumented mismatch
```

One visibility policy feeds both SDK checkpoints

examples/host-kit/visibility.ts · blind-tests app

```
const policy = createVisibilityPolicy({
  repoRoot: checkout.dir,
  hidden: isHiddenFromBlindTester,       // never visible, any access kind
  readable: isBlindReadable,             // optional allowlist for reads
  writable: isBlindWritable,             // where the agent may create files
  shellCommands: BLIND_SHELL,            // executables it may run
  shellTextDeny: BLIND_TEXT_DENY,        // text that reaches hidden material indirectly
  customTools: new Set(["api_contract"]),
  hiddenFeedback: "You are testing a black box.  …  Use api_contract and live HTTP calls.",
});
createSession({ …, onPermissionRequest: policy.onPermissionRequest,
  hooks: { onPreToolUse: policy.onPreToolUse } });
```

onPreToolUse

Raw arguments: catches a grep sweeping a hidden folder.

onPermissionRequest

Parsed request: hidden reads, stray writes, shell commands.

Beyond a hook (Tested)

Write-content checks; per-segment command allowlist.

🔭 Like pi's beforeToolCall, the host holds session state across events.

A blind tester surfaced an API contract mismatch nobody had documented

Does the API behave the way its contract says, judged by someone who never saw the code?

10 / 2

test status: passing / failing on the host's own rerun

Observed 2026-10-26 · TAP output · 18 tool calls, 19 allowed, 2 denied

📄 **Sees only api_contract**

SETUP.md endpoints, bug hints stripped; live HTTP calls.

🙈 **Hidden: source, docs, BUGS.md**

Writes only __tests__/blind/; host serves on 5299.

🐞 **Finding: POST /api/characters → 500**

Referee: BUGS.md #19, Missing Input Validation.

❓ **Finding: seasonId filter ignored**

Referee: undocumented. Docs say seasonId; route reads season_id.

⚖️ A read-only referee labels each failure; the API owner decides which side changes.

Three things no harness can see

Inside a process a tool starts

The harness approves running npm test; what that code does next belongs to the OS.

Scripts it runs have full process access

For a hard wall, run the agent in a container

Anything that isn't a tool call

The harness only decides what the agent asks its tools to do.

Other processes, users, and the network

OS accounts and network rules cover those

Whether the result is right

The harness records what happened; judging it is a separate job.

Tests, reviews, and evidence answer that

A named person decides what ships

Same as pi's find → fd: the harness sees the call and its result; the OS runs the rest.

Every run leaves one log, with every layer joined by sessionId

.runs/my-app/events.jsonl (example)


```
{"layer":"repo-hook","event":"hook.end","sessionId":"909bf25c-…","hook":"preToolUse","success":true}
{"layer":"permission-handler","sessionId":"909bf25c-…","path":"secrets/.env","decision":"deny"}
{"layer":"runtime","event":"tool.complete","sessionId":"909bf25c-…","tool":"view","success":true}
{"layer":"app","event":"session-end","sessionId":"909bf25c-…","toolCalls":18,"allow":16,"deny":2}
```


🔗 One sessionId joins every layer

Repo hooks, your permission handler, and the runtime, lined up for one session.

⚖️ Decisions apart from actions

What was allowed or denied, and separately, what actually ran.

🏷️ Names, rules, and counts

Tool, path, and rule; file contents and command output stay out.

📌 Build reports from the log of what the tools did; the agent's own summary is a claim to check.

PART 4

Own It

Know the controls other people already own, then share your harness with a pre-flight and a named owner.



Surrounding controls

Policy hooks, models, billing, tools, process



Pre-flight

Pins, clean checkouts, bounds, tested rules



Named owner

Every report goes to someone who decides

```
A harness you share becomes a harness someone owns
↳ Which of the five jobs do you take over?
```

Your harness runs inside controls other people already own

What each owner decides, and what your app verifies before you share it

Policy hooks	Load machine-wide before repo hooks; run under disableAllHooks	Platform team
Access	Who uses Copilot, on which surfaces; check your app identity	Org admins
Models	Admins set availability; model auto or a named model is a request	Admins + app
Billing	User or token route follows Copilot policy; BYOK follows provider	Billing admins
Tools + MCP	managedApprovalRequired requests need a human approval path	Admins + app
Process	OS account, container, network: the real boundary for installs	Platform owners

Documented in the hooks reference and Copilot policies; verify each against the identity your app runs as

Before sharing, run a pre-flight and name an owner

✅ Pre-flight before someone else runs it

Pin the SDK; record the runtime version
Every report names the SDK and runtime versions

Start from a clean, pinned checkout
Tool budget, timeout, cleanup in finally

Unit-test every visibility rule
Against real request shapes; set enableFileHooks: true

Confirm auth route, models, billing
For the people who will run it

🧭 How deep to go

A note at session start steers it
→ sessionStart hook

A call must never run, in any harness
→ preToolUse repo or machine policy hook, kept fast

Needs persona, tools, visibility, budget, report
→ SDK host with enableFileHooks: true

Name the owner of every report
Docs owner: friction; API owner: blind failures

🎯 Your move: which of the five jobs do you take over for your workflow, and who reads the report?

THE VALUE RECAP

From Using an Agent to Hosting One

Before

- The agent is a black box you prompt from outside
- One full-access agent for every question
- Rules live in prompts and reviewer memory
- The agent's own summary stands in for the result

After

- Just a loop: you choose where to step in, five known places
- A host per perspective: docs-only, contract-only
- Repo hooks plus a unit-tested host policy
- Host-rerun tests and JSONL evidence joined by sessionId

~170
lines in pi's whole runLoop

24 / 0
fresh-eyes tool calls / source files opened

10 / 2
blind tests passing / failing on the host rerun

Six apps worth hosting, and the jobs that make each one work

Each idea takes over some of the five jobs; the runtime handles the rest

Flaky hunter	Reruns a failing CI test, bisects recent commits, files the culprit	`Loop + Evidence`
Upgrade crew	One session per dependency; each bumps, tests, and keeps only green	`Loop + Control`
Release prep	Drafts notes from merged PRs, bumps the version; a person tags it	`Tools + Control`
Runbook drill	Follows an incident runbook in a sandbox; flags each unclear step	`Context + Evidence`
Exam grader	Runs a student's tests while the answer key stays hidden	`Control`
Issue→release	Plan, code, review, ship: one session each, a person between stages	`All five`

The tag names the job you write code for; start with that one

✅ WHAT YOU CAN DO TODAY

Today

- Watch your agent work and name the job behind each step
- Spot the loop: ask the model, run the tool, feed back the result
- List the rules you rely on that live only in a prompt

This Week

- Read one open-source loop, like pi's runLoop, end to end
- Ask what your agent can see, write, and run, and who decided
- Move one prompt-only rule into a hook and watch it hold

This Month

- Pick one build or release chore and name the jobs you would own
- Sketch a small host: one session, one custom tool, one rule
- Ask who would read its report, and what they would want in it

🔑 Key Takeaway

A harness is just a loop: pick the place you want to step in, and start there.